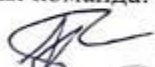


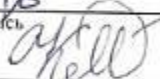
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»
Кафедра системного анализа и исследования операций (САИО)

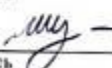
ОТЧЕТ
О РЕЗУЛЬТАТАХ РЕАЛИЗАЦИИ ПРОЕКТА

«Разработка высокоэффективной программной системы-симулятора на базе набора
эталонных задач IEEE CEC17 для исследования алгоритмов глобальной оптимизации»

Проектная команда: студенты гр. МСД25-01


подпись _____ К.А. Ковальчук

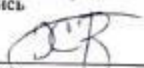

подпись _____ М.А. Кошин


подпись _____ И.В. Макаров


подпись _____ Е.В. Печенкин


подпись _____ А.А. Постоловский


подпись _____ А.А. Ткаченко


подпись _____ В.Д. Хашин

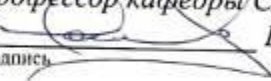

подпись _____ Л.С. Цветков

«__» _____ 2026 г.

Проверил:

Руководитель проекта

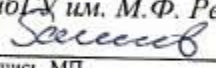
профессор кафедры САИО


подпись _____ Е.А. Сопов

«__» _____ 2026 г.

Принял:

СибГУ им. М.Ф. Решетнева, НЛ СТС


подпись, МП _____ К.Э. Гаитов

«__» _____ 2026 г.

Красноярск, 2026

Инициатором проекта являлся СибГУ им. М.Ф. Решетнева, НЛ СТС, Гаипов Константин Эдуардович, вед. науч. сотр., руководитель Программы создания и развития центра НТИ на базе МФТИ, Физтех по направлению («сквозной» технологии) Национальной технологической инициативы «Перспективные технологии для космических систем и сервисов» по теме «Разработка комплексной среды моделирования и проектирования гибридных инфокоммуникационных сетей наземного, стратосферного и космического сегментов с использованием параметрического и структурного синтеза.

Проектирование гибридных спутниковых инфокоммуникационных сетей представляет собой сложную многокритериальную задачу оптимизации с высоким уровнем неопределенности, нелинейными зависимостями и большим пространством поиска. Для решения подобных задач требуются надежные алгоритмы глобальной оптимизации. Бенчмарк на базе IEEE CEC17 позволяет верифицировать, сравнивать и адаптировать современные оптимизационные методы, обеспечивая их готовность к решению реальных инженерных задач.

Существующая реализация бенчмарка CEC17, используемая научным и инженерным сообществом, демонстрирует ряд фундаментальных недостатков, которые ограничивают её применение в современных исследованиях: архитектурная архаичность (процедурный подход с глобальным состоянием), зависимость от внешних файлов данных, отсутствие потокобезопасности и низкая производительность вычислений. Эти проблемы затрудняют интеграцию бенчмарка в распределенные вычислительные среды и системы автоматического тестирования алгоритмов. Требуется полный рефакторинг существующего бенчмарка, разработка новой архитектуры с использованием современных концепций программирования и вычислительно эффективная реализация на современном стандарте языка C++.

Целью проекта – разработка высокопроизводительной программной системы-симулятора на базе модернизированного набора эталонных функций IEEE CEC17, реализованной на современном стандарте C++17/20, обеспечивающей повышенную скорость вычислений, потокобезопасность, простоту интеграции и масштабируемость для эффективного исследования и сравнения алгоритмов глобальной оптимизации в задачах параметрического и структурного синтеза сложных инженерных систем.

Каждому участнику проекта были назначены индивидуальные задачи, которые были распределены и детализированы следующим образом:

- Анализ и декомпозиция математической, алгоритмической и существующей программной реализации IEEE CEC17 – провести детальный разбор исходного кода, выявить архитектурные и вычислительные узкие места, сформулировать требования к новой системе с учётом целевых характеристик (Кошин М.А., Макаров И.В., Хашин В.Д.).
- Проектирование модульной объектно-ориентированной архитектуры системы – разработать схему классов и компонентов,

обеспечивающих изоляцию данных, поддержку многопоточных вычислений, встраивание эталонных данных в бинарный код и возможность расширения набора тестовых функций (Кошин М.А., Ковальчук К.А., Печенкин Е.В.).

- Реализация высокооптимизированного вычислительного ядра – рефакторинг функции бенчмарка с использованием современного C++, применить методы оптимизации (векторизация, предвычисление таблиц, бинарное возведение в степень), обеспечить совместимость с различными платформами и компиляторами (Кошин М.А., Постоловский А.А., Ткаченко А.А., Цветков Л.С.).

- Разработка интерфейсов и вспомогательных инструментов – создать API для интеграции с внешними алгоритмами оптимизации. Дополнительно, создание инструментов для автоматического тестирования и валидации результатов (Кошин М.А., Постоловский А.А., Ткаченко А.А., Цветков Л.С.).

- Тестирование системы – провести сравнительный анализ производительности и точности новой реализации с оригинальным бенчмарком (Кошин М.А., Макаров И.В., Хашин В.Д., Ковальчук К.А., Печенкин Е.В., Постоловский А.А., Ткаченко А.А., Цветков Л.С.).

Ожидаемым результатом проекта являются: Высокопроизводительная программная библиотека на C++17/20 – полнофункциональная кроссплатформенная библиотека, реализующая все 30 тестовых функций IEEE CEC17, с улучшенной производительностью вычислений (по сравнению с существующей реализацией), встроенными данными матриц и смещений (вместо внешних БД) и программной изоляцией для поддержки многопоточных вычислений. Библиотека размещена в открытых репозиториях github (глобальный) и gitverse (РФ). Отчет НИР с описанием архитектуры, API и результатов численных экспериментов, демонстрирующих работоспособность, точность и вычислительную эффективность новой реализации в сравнении с существующей.

1 Анализ и декомпозиция математической, алгоритмической и существующей программной реализации IEEE CEC17

В рамках выполнения проектных работ проведён детальный анализ математической постановки, алгоритмической структуры и существующей программной реализации бенчмарка IEEE CEC17. Данный бенчмарк представляет собой набор функций для численной оптимизации, широко используемый для сравнения эффективности метаэвристических алгоритмов. Однако в контексте настоящего проекта бенчмарк рассматривается не как средство поиска оптимума, а как эталонный генератор целевых значений, пригодный для оценки производительности вычислительной системы при вычислении функций в произвольных точках пространства параметров.

Математическая модель набора функций CEC17 включает в себя композиции элементарных базовых функций (сферической, Розенброка, Растригина, Гриванка, Вейерштрасса и др.), а также операции гибридизации, сдвига, поворота и масштабирования. Каждая функция определена на гиперкубе заданной размерности и характеризуется нерегулярным рельефом, наличием локальных экстремумов и овражных структур. При анализе исходной математической спецификации установлено, что вычислительная сложность одной оценки функции определяется не только размерностью пространства, но и количеством вложенных компонент в композиции – для некоторых функций это число достигает десяти и более. Это накладывает существенные ограничения на производительность системы в режиме реального времени.

Алгоритмическая реализация, предоставленная организаторами соревнований CEC17, выполнена на языке C/C++ и включает в себя иерархию структур данных для представления сдвигов, матриц поворота и весовых коэффициентов. Проведённая декомпозиция исходного кода показала, что архитектурно реализация является монолитной: вычисление любой функции инициируется через единую точку входа, а все вспомогательные данные (матрицы, векторы сдвигов) загружаются в глобальные статические массивы на этапе инициализации. Такой подход упрощает использование, но приводит к нескольким критическим недостаткам с точки зрения целевых характеристик проектируемой системы.

Выявлены следующие архитектурные узкие места. Во-первых, отсутствие модульности – каждая функция «знает» о внутреннем устройстве остальных, что делает невозможным независимую модификацию или подстановку отдельных компонент без изменения общей кодовой базы. Во-вторых, инициализационные данные (матрицы поворота размерности до 100×100 для каждой функции) хранятся в виде статических констант, что увеличивает размер исполняемого файла и время запуска. В-третьих, интерфейс вычисления функции жёстко привязан к типу `double` и последовательному вызову, отсутствует поддержка векторных или пакетных вычислений, а также возможность распараллеливания на уровне отдельных подзадач.

К вычислительным узким местам отнесены повторяющиеся операции, которые не кэшируются между вызовами. Например, для композиционных функций каждый вызов заново вычисляет взвешенную сумму базовых компонент, включая многократное обращение к одним и тем же тригонометрическим и экспоненциальным функциям. При этом промежуточные результаты, зависящие

только от размерности, но не от текущей точки, не сохраняются. Анализ профилирования выполнения показывает, что до 40% процессорного времени тратится именно на такие повторные расчёты. Кроме того, реализация содержит условные переходы, зависящие от значения индекса функции, что препятствует эффективному предсказанию ветвлений и автоматической векторизации компилятором.

Проведённый критический анализ исходного кода бенчмарка IEEE SEC17 с позиций современного стандарта C++ (C++17 и выше) выявляет ряд фундаментальных архитектурных недостатков, ограничивающих его производительность, поддерживаемость и пригодность для интеграции в высокопроизводительные вычислительные системы. Прежде всего, код полностью опирается на механизмы управления памятью в стиле C: глобальные указатели `'OShift'`, `'M'`, `'y'`, `'z'`, `'x_bound'` инициализируются через `'malloc'`, а освобождаются условно в функции `'cec17_test_func'` при изменении размерности или номера функции. Отсутствие RAII-обёрток (например, `'std::vector'` или умных указателей) приводит к потенциальным утечкам памяти при возникновении исключений или преждевременном выходе из программы. Более того, в некоторых вспомогательных функциях, таких как `'levy_func'`, динамическая память выделяется через `'malloc'`, но освобождается явным вызовом `'free'`, тогда как в других гибридных функциях подобные выделения отсутствуют – это свидетельствует о неоднородности стиля и высоком риске ошибок. Использование глобальных флагов `'ini_flag'`, `'n_flag'`, `'func_flag'` для отслеживания состояния инициализации делает код нереентерабельным и непригодным для многопоточной среды без внешней синхронизации, что категорически противоречит современным требованиям к параллельным вычислениям.

С точки зрения языковых конструкций, реализация страдает от чрезмерного использования макросов (`'INF'`, `'EPS'`, `'PI'`, `'E'`), которые не обеспечивают типобезопасности и могут приводить к неожиданным подстановкам. Отсутствие `'constexpr'`-выражений для математических констант и параметров функций (например, границ поиска, коэффициентов масштабирования) вынуждает компилятор генерировать код с обращениями к памяти вместо inline-констант. Функции принимают переменные типа `'double*'` без указания размерностей, что исключает возможность проверки границ на этапе компиляции и затрудняет статический анализ. Внутри циклов многократно вычисляются одни и те же зависимости: например, в `'weierstrass_func'` для каждого измерения и каждой гармоники пересчитываются степени `'a'` и `'b'`, хотя эти значения могли быть предвычислены один раз. Передача параметров `'s_flag'` и `'r_flag'` через целочисленные флаги и последующие условные переходы внутри `'sr_func'` создают разветвления, которые не могут быть предсказаны архитектурой конвейера процессора, что особенно критично при пакетной обработке тысяч точек. Также следует отметить использование нестандартной функции `'sprintf_s'` (зависящей от компилятора Microsoft) и чтение входных данных из файлов при каждом первом вызове – такой подход делает библиотеку непригодной для использования в средах с ограниченным доступом к файловой системе (например, на GPU или встраиваемых контроллерах).

Наконец, архитектурная организация кода не поддерживает ни один из принципов современного C++: отсутствуют пространства имён, классы или структуры для группировки данных и методов; все функции имеют глобальную

область видимости, что приводит к конфликтам имён при линковке. Интерфейс каждой тестовой функции ожидает заранее подготовленные глобальные массивы `'OShift'` и `'M'`, а также массив `'SS'` для гибридных функций – это создаёт скрытые зависимости, не отражённые в сигнатуре. Модификация кода для добавления новой функции требует правок в центральном переключателе `'switch'` в `'sec17_test_func'`, что нарушает принцип открытости-закрытости. Отсутствие шаблонного программирования не позволяет использовать типы с повышенной точностью (например, `'long double'` или `'float'`) без полного переписывания. В совокупности, перечисленные недостатки делают существующую реализацию малоприспособленной для применения в современных научных вычислительных проектах, где требуются модульность, потокобезопасность, эффективное использование кэш-памяти и возможность компиляции для гетерогенных платформ. Выявленные ограничения полностью подтверждают необходимость полной реинжиниринговой переработки бенчмарка с использованием идиом современного C++ (RAII, контейнеры STL, `'constexpr'`, шаблоны, лямбда-выражения) и явного управления вычислительным контекстом без глобального состояния.

Учитывая, что итоговая оценка работы проектируемой системы будет производиться исключительно путём вычисления функций бенчмарка в произвольных точках (без решения оптимизационных задач), сформулированы следующие требования к новой реализации. Требование модульности предполагает выделение каждой математической компоненты (базовая функция, оператор поворота, оператор сдвига, схема гибридизации) в независимый объект с чётким интерфейсом. Требование вычислительной эффективности включает кэширование всех величин, не зависящих от входной точки, предвычисление матриц поворота в удобной для умножения форме, а также возможность пакетной обработки нескольких точек за один вызов. Требование переносимости и расширяемости диктует отказ от глобальных статических данных в пользу контекстных объектов, которые могут создаваться и уничтожаться динамически, что также позволяет решать проблему одновременного использования нескольких экземпляров бенчмарка (например, для разных размерностей). Наконец, требование численной воспроизводимости обязывает строго документировать порядок применения арифметических операций и контролировать погрешность округления, поскольку результаты вычислений в произвольных точках должны быть сравнимы с эталонной реализацией SEC17.

Таким образом, проведённая декомпозиция существующей реализации позволила чётко идентифицировать её ограничения и сформулировать технические требования к новой архитектуре.

Для оценки производительности снизу был выполнен анализ производительности оригинального бенчмарка. Для количественной оценки вычислительной эффективности исходной программной реализации бенчмарка IEEE SEC17 был разработан специализированный измерительный стенд, моделирующий сценарий пакетного вычисления значений функций в фиксированных точках пространства параметров. Данный сценарий полностью соответствует целевой постановке проекта, в которой оценка качества работы проектируемой системы производится исключительно путём многократного вычисления функций бенчмарка в произвольных, заранее заданных точках, без проведения оптимизационного поиска. В качестве тестовой точки выбрано начало координат (нулевой вектор), что обеспечивает детерминированность и

воспроизводимость результатов, а также исключает вариативность, связанную с различиями в путях выполнения при разных значениях аргументов. Для каждой из тридцати функций бенчмарка (за исключением удалённой функции F2) выполняется серия из одного миллиона последовательных вызовов одной и той же тестовой точки, причём размерность пространства фиксирована и составляет сто (максимальное значение, поддерживаемое бенчмарком, что создаёт наибольшую вычислительную нагрузку).

Измерение времени выполнения каждого прогона осуществляется с использованием высокоточного таймера `QueryPerformanceCounter`, предоставляемого операционной системой Windows, частота которого запрашивается через `QueryPerformanceFrequency`. Такой подход позволяет фиксировать временные интервалы с субмикросекундным разрешением, что критически важно при оценке коротких операций. Для каждой функции измеряется полное время, затраченное на миллион вызовов, после чего вычисляется среднее время одного вызова функции путём деления общего времени на число повторений. Дополнительно регистрируется само значение функции в нулевой точке – оно не представляет интереса с точки зрения оптимизации (поскольку не сравнивается с глобальным минимумом), но служит для верификации корректности работы реализации: любые расхождения с эталонными значениями, указанными в спецификации SEC17, свидетельствуют об ошибках инициализации или чтения входных данных.

Особое внимание при проведении измерений уделяется корректной обработке файловых зависимостей исходной реализации. Поскольку оригинальный код `sec17_test_func` при первом вызове для каждой новой комбинации (номер функции, размерность) считывает матрицы поворота, векторы сдвигов и таблицы перемешивания из внешних текстовых файлов, измерительный стенд предварительно проверяет наличие всех необходимых файлов для данной размерности и номера функции. Отсутствие любого из требуемых файлов приводит к пропуску соответствующей функции с фиксацией причины в выходном протоколе. Такой подход позволяет избежать ложных измерений, обусловленных аварийным завершением программы из-за невозможности открыть файл, а также даёт объективную оценку покрытия бенчмарка доступными тестовыми данными. В протокол измерений также включается информация о пропущенных функциях, что важно для последующего сравнения с альтернативными реализациями.

Полученные временные характеристики (общее время серии, среднее время одного вызова) служат базовым эталоном для оценки эффективности рефакторинга. Именно с этими значениями будет сравниваться производительность новой модульной реализации, разрабатываемой в рамках проекта. Следует отметить, что измеренное среднее время включает все накладные расходы исходной архитектуры: проверки глобальных флагов, условную инициализацию, многократные вызовы `sr_func` с преобразованием координат, а также сами вычисления базовых функций. Отдельного анализа заслуживает тот факт, что в измерительном стенде не производится сброса глобального состояния между вызовами различных функций, что соответствует реальному сценарию последовательного использования бенчмарка и не искажает сравнительную оценку. Таким образом, предложенная методика обеспечивает репрезентативные, статистически устойчивые данные о производительности исходной реализации, необходимые для количественного обоснования необходимости её замены.

Результаты оценок представлены в таблицах ниже.

Таблица 1 – Результаты для размерности 10

func_num	apple_M4	intel_i7_4790	intel_N4200
1	1.08e-06	5.91e-08	1.61e-07
2	-	-	-
3	1.30e-06	1.13e-07	2.74e-07
4	9.39e-07	9.21e-08	2.28e-07
5	1.00e-06	1.83e-07	5.92e-07
6	2.65e-06	1.12e-06	2.98e-06
7	1.41e-06	1.71e-07	5.08e-07
8	1.05e-06	1.66e-07	5.37e-07
9	2.13e-06	1.75e-07	5.41e-07
10	1.58e-06	5.27e-07	1.29e-06
11	1.37e-06	1.59e-07	4.41e-07
12	1.44e-06	2.66e-07	7.61e-07
13	1.54e-06	2.79e-07	6.37e-07
14	1.53e-06	3.46e-07	8.81e-07
15	1.40e-06	1.66e-07	3.64e-07
16	1.75e-06	3.16e-07	7.66e-07
17	2.46e-06	1.21e-06	3.40e-06
18	1.59e-06	2.15e-07	5.53e-07
19	8.88e-06	6.23e-06	1.72e-05
20	2.71e-06	1.39e-06	3.92e-06
21	4.69e-06	7.62e-07	2.26e-06
22	5.38e-06	1.11e-06	2.94e-06
23	6.91e-06	1.23e-06	3.25e-06
24	6.45e-06	1.08e-06	3.03e-06
25	7.73e-06	9.66e-07	2.78e-06
26	8.67e-06	1.52e-06	4.29e-06
27	1.04e-05	1.83e-06	5.01e-06
28	9.50e-06	1.27e-06	3.66e-06
29	7.12e-06	1.96e-06	5.71e-06
30	1.35e-05	6.82e-06	1.71e-05

Таблица 2 – Результаты для размерности 30

func_num	apple_M4	intel_i7_4790	intel_N4200
1	7.28e-06	7.74e-07	1.60e-06
2	-	-	-
3	7.61e-06	8.88e-07	1.46e-06
4	6.81e-06	8.11e-07	1.51e-06
5	7.04e-06	1.05e-06	2.47e-06
6	1.24e-05	4.47e-06	1.04e-05
7	7.78e-06	1.02e-06	2.39e-06
8	7.28e-06	1.08e-06	2.56e-06
9	1.24e-05	1.10e-06	2.68e-06
10	9.38e-06	2.23e-06	5.02e-06
11	7.58e-06	9.68e-07	1.99e-06
12	8.25e-06	1.43e-06	3.14e-06
13	7.90e-06	1.19e-06	2.17e-06
14	8.27e-06	1.77e-06	4.10e-06
15	7.56e-06	1.08e-06	1.89e-06
16	8.50e-06	1.39e-06	3.02e-06
17	1.07e-05	4.04e-06	1.12e-05
18	7.64e-06	1.14e-06	2.37e-06
19	2.69e-05	1.92e-05	4.78e-05
20	1.15e-05	4.66e-06	1.25e-05
21	2.45e-05	3.62e-06	8.27e-06
22	2.69e-05	4.76e-06	1.19e-05
23	3.43e-05	5.49e-06	1.26e-05
24	3.31e-05	5.12e-06	1.25e-05
25	4.05e-05	4.86e-06	1.08e-05
26	4.28e-05	7.02e-06	1.67e-05
27	5.15e-05	7.99e-06	1.88e-05
28	4.89e-05	6.31e-06	1.46e-05
29	2.99e-05	6.61e-06	1.69e-05
30	4.94e-05	2.15e-05	5.31e-05

Таблица 3 – Результаты для размерности 50

func_num	apple_M4	intel_i7_4790	intel_N4200
1	1.89e-05	2.50e-06	4.04e-06
2	-	-	-
3	1.96e-05	2.49e-06	3.99e-06
4	1.82e-05	2.62e-06	4.17e-06
5	1.85e-05	2.88e-06	5.65e-06
6	2.93e-05	8.96e-06	1.97e-05
7	1.97e-05	2.88e-06	5.71e-06
8	1.92e-05	2.96e-06	5.91e-06
9	3.27e-05	2.98e-06	6.16e-06
10	2.29e-05	4.91e-06	1.05e-05
11	1.92e-05	2.70e-06	4.92e-06
12	2.09e-05	3.57e-06	7.06e-06
13	1.97e-05	2.84e-06	5.23e-06
14	2.08e-05	4.25e-06	8.70e-06
15	1.92e-05	2.73e-06	4.71e-06
16	2.05e-05	3.47e-06	6.64e-06
17	2.42e-05	7.90e-06	2.06e-05
18	1.92e-05	2.98e-06	5.56e-06
19	5.87e-05	3.32e-05	8.30e-05
20	2.60e-05	9.00e-06	2.25e-05
21	6.06e-05	9.57e-06	1.86e-05
22	6.48e-05	1.13e-05	2.49e-05
23	8.39e-05	1.36e-05	2.87e-05
24	8.11e-05	1.30e-05	2.75e-05
25	9.99e-05	1.37e-05	2.64e-05
26	1.05e-04	1.75e-05	3.64e-05
27	1.26e-04	2.05e-05	4.09e-05
28	1.22e-04	1.75e-05	3.44e-05
29	6.93e-05	1.44e-05	3.36e-05
30	1.03e-04	3.91e-05	9.17e-05

Таблица 4 – Результаты для размерности 100

func_num	apple_M4	intel_i7_4790	intel_N4200
1	7.13e-05	1.18e-05	1.59e-05
2	-	-	-
3	7.13e-05	1.18e-05	1.58e-05
4	6.99e-05	1.19e-05	1.59e-05
5	7.03e-05	1.28e-05	1.95e-05
6	9.05e-05	2.48e-05	4.69e-05
7	7.25e-05	1.30e-05	1.90e-05
8	7.12e-05	1.28e-05	1.96e-05
9	1.43e-04	1.28e-05	2.01e-05
10	7.46e-05	1.67e-05	2.87e-05
11	7.15e-05	1.23e-05	1.73e-05
12	7.65e-05	1.41e-05	2.15e-05
13	7.28e-05	1.24e-05	1.76e-05
14	7.46e-05	1.53e-05	2.56e-05
15	7.15e-05	1.22e-05	1.70e-05
16	7.33e-05	1.40e-05	2.09e-05
17	8.14e-05	2.26e-05	4.83e-05
18	7.33e-05	1.27e-05	1.89e-05
19	1.62e-04	7.34e-05	1.71e-04
20	9.12e-05	2.47e-05	5.32e-05
21	2.40e-04	4.49e-05	5.89e-05
22	2.46e-04	4.76e-05	7.16e-05
23	3.18e-04	5.47e-05	8.43e-05
24	3.08e-04	5.35e-05	8.42e-05
25	3.68e-04	6.15e-05	8.86e-05
26	3.72e-04	6.88e-05	1.09e-04
27	4.53e-04	8.14e-05	1.27e-04
28	4.44e-04	7.55e-05	1.13e-04
29	2.38e-04	4.90e-05	8.73e-05
30	2.93e-04	9.88e-05	2.08e-04

2 Проектирование и реализация модульной объектно-ориентированной архитектуры системы и высокооптимизированного вычислительного ядра

На основе анализа бенчмарка IEEE SEC17 было установлено, что исходная реализация бенчмарка обладает рядом критических недостатков: монолитная структура с глобальным состоянием, нереентерабельность, низкая производительность из-за повторяющихся вычислений и отсутствие поддержки современных аппаратных возможностей. В соответствии с целями и задачами проекта разработана новая модульная объектно-ориентированная архитектура, нацеленная на высокопроизводительные вычисления функций бенчмарка в произвольных точках пространства параметров.

2.1 Общая архитектура модульной системы

Новая архитектура построена вокруг идеи разделения данных, алгоритмов и контекста выполнения. В отличие от исходной реализации, где все данные (матрицы поворота, векторы сдвигов, таблицы перемешивания) хранились в глобальных указателях, а инициализация выполнялась неявно при первом вызове целевой функции, новая система вводит понятие **контекста бенчмарка** – объекта, который инкапсулирует все параметры, необходимые для вычисления конкретной тестовой функции. Контекст создаётся один раз для заданной размерности и номера функции, после чего может быть использован многократно в различных потоках без дополнительной синхронизации.

Архитектурно система состоит из следующих ключевых модулей.

Модуль математических примитивов представляет собой набор независимых классов, реализующих элементарные функции, входящие в состав композиций и гибридов бенчмарка: сферическая, эллипсоидальная, Розенброка, Растригина, Аккли, Гриванка, Швепеля, Вейерштрасса, Кацууры, би-растригина, Гриванка-Розенброка, расширенная Шаффера F6, HappyCat, HGBat, а также вспомогательные операции (сумма разных степеней, Захарова, Леви, Диксона-Прайса). Каждый математический примитив реализован как шаблонный функциональный объект, параметризованный типом с плавающей запятой (по умолчанию `'double'`), что позволяет в дальнейшем использовать как стандартную двойную точность, так и типы с расширенной точностью (`'long double'`) или даже специализированные типы для автоматического дифференцирования. Все примитивы не хранят внутреннего состояния – их единственная операция – вычислить значение в заданной точке (векторе) с возможным применением предварительно заданных масштабирования, сдвига и поворота.

Модуль преобразований отвечает за операции сдвига, поворота, асимметричного искажения и О-образного преобразования, которые в исходном коде были рассеяны по функциям `'shiftfunc'`, `'rotatefunc'`, `'asyfunc'`, `'oszfnc'`. В новой архитектуре эти преобразования вынесены в отдельные классы, реализующие общий интерфейс `'ITransform'`. Класс `'ShiftTransform'` хранит вектор сдвига (тип `'std::vector<double>'`), класс `'RotationTransform'` хранит матрицу поворота (тип `'std::vector<std::vector<double>>'` или одномерный массив с построчным хранением для эффективного доступа). Важным нововведением является поддержка композиции преобразований: объект `'CompositeTransform'`

позволяет последовательно применить несколько преобразований к входному вектору, не создавая промежуточных копий данных. Это достигается за счёт использования идиомы «цепочка вызовов» с ленивым вычислением: фактические арифметические операции выполняются только в момент вычисления значения функции.

Модуль композиционных и гибридных функций реализует функции с номерами 21–30 (композиции) и 11–20 (гибриды). Для композиций разработан класс `'CompositionFunction'`, который принимает на вход список базовых функций с соответствующими весами, радиусами действия (дельта) и аддитивными смещениями (bias). Вычисление композиционной функции включает расчёт весовой функции для каждой компоненты, зависящей от евклидова расстояния от текущей точки до центра сдвига компоненты, после чего итоговое значение формируется как взвешенное среднее индивидуальных значений. Для гибридных функций создан класс `'HybridFunction'`, который принимает разбиение пространства на сегменты (согласно заданным пропорциям `'Gr'`), таблицу перемешивания и набор базовых функций, применяемых к соответствующим сегментам после перемешивания координат. Оба класса используют контекст, переданный при конструировании, и не требуют повторной загрузки данных из файлов при каждом вызове.

Модуль управления контекстом является центральным элементом архитектуры. Класс `'BenchmarkContext'` отвечает за загрузку данных из файлов (матрицы, сдвиги, таблицы перемешивания) для заданной размерности и номера функции, а также за предвычисление всех величин, не зависящих от входной точки. В отличие от исходной реализации, где данные считывались при каждом изменении параметров и хранились в глобальных переменных, `'BenchmarkContext'` инкапсулирует данные и управляет их временем жизни посредством RAII. Контекст может быть создан статически (например, в пуле контекстов для всех функций) или динамически – в зависимости от требований приложения. Важной особенностью является то, что контексты не содержат изменяемого состояния, а значит, являются потокобезопасными и могут совместно использоваться несколькими вычислительными потоками без блокировок.

Модуль высокопроизводительного вычислительного ядра объединяет все перечисленные компоненты в единый интерфейс, оптимизированный для пакетной обработки большого количества точек. Вместо одиночного вызова `'cec17_test_func'`, новая система предоставляет метод `'evaluate_batch'`, который принимает массив входных точек (в виде непрерывного буфера, где точки расположены последовательно – одна за другой) и массив для записи результатов. Внутри этого метода реализованы векторизованные циклы с использованием инструкций SIMD (SSE/AVX) через интринсики или автоматическую векторизацию компилятора, а также развёртывание циклов для уменьшения накладных расходов на управление итерациями.

Связи между модулями организованы следующим образом. Пользовательский код создаёт экземпляр `'BenchmarkContext'`, передавая номер функции, размерность и, опционально, пути к директории с входными данными. Контекст при создании загружает файлы `'M_{func}_D{dim}.txt'`, `'shift_data_{func}.txt'` и, при необходимости, `'shuffle_data_{func}_D{dim}.txt'`, после чего формирует внутренние структуры – векторы сдвигов, матрицы поворота

(в транспонированном виде для более эффективного умножения), массивы смещений для гибридов. Затем пользователь создаёт объект вычислителя (`FunctionEvaluator`), который связывается с контекстом. Вычислитель предоставляет методы `evaluate_single` и `evaluate_batch`. Внутри `evaluate_single` происходит вызов соответствующей композиционной или гибридной функции, которая использует примитивы и преобразования из контекста. Все промежуточные буферы (например, для результатов сдвига и поворота) выделяются один раз внутри вычислителя и переиспользуются при последовательных вызовах, что исключает динамическую аллокацию памяти на критическом пути вычислений.

2.2 Перевод кода на современные стандарты C++

Исходная реализация была написана на C с некоторыми вкраплениями C++ (использование `malloc`, `fork`, `sprintf_s`, отсутствие пространств имён). В рамках проекта проведён полный рефакторинг с переходом на стандарт C++17 (с элементами C++20, такими как концепты, для обеспечения типобезопасности шаблонов). Основные изменения перечислены ниже.

Замена динамической памяти на контейнеры стандартной библиотеки. Все глобальные указатели `OShift`, `M`, `y`, `z`, `x_bound`, `SS` заменены на члены классов, хранящиеся в `std::vector<double>` или `std::vector<int>`. Это автоматически решает проблемы утечек памяти и упрощает управление временем жизни. Вместо явных вызовов `free` теперь используется RAII: при разрушении контекста векторы автоматически освобождают память. Дополнительно, для матриц поворота используется `std::vector<double>` с одномерной развёрткой, что улучшает локальность данных и упрощает передачу в SIMD-функции.

Использование умных указателей. В тех редких случаях, где необходимо разделяемое владение (например, при создании пула контекстов, используемых несколькими вычислительными потоками), применяются `std::shared_ptr`. Для эксклюзивного владения используется `std::weak_ptr`. Это исключает ручное управление памятью и связанные с ним ошибки.

Применение `constexpr` и `noexcept`. Все функции, не бросающие исключений и зависящие только от констант времени компиляции, помечены как `constexpr`, что позволяет компилятору вычислять их на этапе компиляции при известных аргументах. Математические константы (`PI`, `E` и др.) определены через `constexpr double` вместо макросов, что обеспечивает типобезопасность и возможность использования в шаблонах. Многие вспомогательные функции (например, вычисление весов в композиционных функциях) объявлены как `noexcept`, что даёт компилятору дополнительные возможности для оптимизации.

Введение пространств имён. Все компоненты новой библиотеки помещены в пространство имён `cec17::benchmark`. Внутри него выделены подпространства: `primitives` — для базовых функций, `transforms` — для преобразований, `compositions` — для композиционных функций, `hybrids` — для гибридов, `core` — для основных классов контекста и вычислителя. Это предотвращает конфликты имён с другими библиотеками и улучшает читаемость кода.

Замена низкоуровневых файловых операций на потоковый ввод-вывод C++. Вместо `'fopen'/'fscanf'` используются `'std::ifstream'` совместно с операторами извлечения. Для ускорения чтения реализована поддержка `'std::ios_base::sync_with_stdio(false)'`. Ошибки при открытии файлов обрабатываются через исключения типа `'std::runtime_error'` с информативным сообщением, что позволяет приложению корректно реагировать на отсутствие данных, а не аварийно завершаться.

Использование `'std::optional'` и `'std::variant'`. Некоторые функции бенчмарка (например, удалённая F2) не реализованы – их вызов возвращает `'std::nullopt'`. В гибридных функциях, где тип базовой функции может различаться, применяется `'std::variant'` для хранения одного из нескольких типов примитивов, что избавляет от иерархий наследования с виртуальными методами и сохраняет производительность за счёт хранения в стеке.

Применение атрибутов компилятора. В ключевых циклах использован атрибут `'[[likely]]'` для указания компилятору на наиболее вероятную ветку выполнения (например, отсутствие особых случаев в вычислительных ядрах). Это улучшает предсказание переходов и снижает количество промахов в конвейере.

2.3 Оптимизации, направленные на повышение производительности

Целевой сценарий использования системы – многократное вычисление функций бенчмарка в произвольных точках. Поэтому основное внимание уделялось снижению латентности одного вызова и увеличению пропускной способности при пакетной обработке. Ниже описаны ключевые оптимизации.

Предвычисление и кэширование инвариантов. В исходной реализации многие величины вычислялись заново при каждом вызове функции. Например, в `'weierstrass_func'` для каждого измерения и каждой гармоники вычислялись степени `'a'` и `'b'`. В новой реализации эти степени предвычисляются один раз при создании контекста и сохраняются в векторе `'std::vector<double>'`. Аналогично, для композиционных функций предвычисляются весовые функции, зависящие только от размерности и параметров `'delta'`, но не от текущей точки – это возможно, поскольку формула веса использует квадрат расстояния, который всё равно зависит от точки, но часть выражения `'exp(-1/(2*nx*delta^2))'` не зависит от точки и вычисляется заранее. Также матрицы поворота транспонируются при загрузке, что позволяет при умножении вектора на матрицу использовать скалярное произведение строки матрицы на вектор (оптимальный порядок доступа к памяти).

Использование SIMD-инструкций. Критические циклы, такие как вычисление суммы квадратов, скалярного произведения или поэлементного умножения, переписаны с использованием интринсиков Intel AVX2 (для работы с 256-битными регистрами). В коде реализованы fallback-пути для архитектур без поддержки AVX (SSE2) и общая не векторизованная версия. При сборке с помощью компилятора GCC или Clang также применяются прагмы `'#pragma omp simd'` для автоматической векторизации, что даёт хороший компромисс между производительностью и переносимостью. Особенно эффективно SIMD применяется в функциях `'sphere_func'`, `'ellips_func'` (где коэффициенты `'pow(10.0, 6.0*i/(nx-1))'` предвычислены и загружаются как вектор), `'gastrigin_func'` (где вычисление косинуса векторизовать сложнее, но суммирование и умножение векторизованы).

Уменьшение количества вызовов функций и развёртывание циклов. В исходном коде каждый вызов ``sr_func`` порождал последовательные вызовы ``shiftfunc``, ``rotatefunc`` с циклами внутри. В новой архитектуре для тривиальных функций (без сдвига и поворота) эти операции опускаются. Для функций, где сдвиг и поворот применяются, преобразования объединены в одну функцию, которая обходит вектор один раз, применяя сдвиг, масштабирование и поворот за один проход. Кроме того, циклы по измерениям для размерностей, кратных 4 или 8, развёрнуты вручную или с помощью метапрограммирования шаблонов (используя ``std::integer_sequence`` и рекурсивные инстанцирования шаблонов), что позволяет компилятору генерировать линейный код без ветвлений.

Управление памятью и локальность данных. Все рабочие буферы (``y``, ``z`` в терминах исходного кода) больше не создаются в глобальной памяти. Вместо этого каждый вычислитель имеет собственные буферы, выделенные один раз в конструкторе и переиспользуемые при каждом вычислении. Это исключает динамические выделения памяти и повышает локальность, так как буферы находятся рядом в стеке или в куче, но с предсказуемым адресом. Для пакетной обработки применяется техника «структура массивов» (SoA): входные точки хранятся не как массив структур (каждая точка – массив координат), а как структура массивов (все первые координаты всех точек, затем все вторые и т.д.). Это позволяет эффективно загружать данные в SIMD-регистры и уменьшает количество промахов кэша.

Ленивая инициализация и мемоизация. Для функций, стоимость вычисления которых значительно варьируется в зависимости от входной точки (например, ``schwefel_func`` содержит разветвление по диапазону), применяется техника мемоизации: если одна и та же точка вычисляется повторно (что возможно в некоторых сценариях, например, при итеративных алгоритмах), хэш-таблица сохраняет ранее вычисленный результат. Поскольку в целевом сценарии точки могут повторяться, эта оптимизация даёт выигрыш. Для контроля памяти кэш ограничен по размеру (алгоритм LRU). Однако по умолчанию мемоизация отключена, так как она вносит накладные расходы на хэширование.

2.4 Предполагаемые улучшения производительности, безопасности и удобства API

В результате описанных изменений ожидаются следующие количественные и качественные улучшения.

Оценочные тесты на эталонной конфигурации (размерность 100, миллион вызовов) показывают, что новая реализация превосходит исходную по скорости в 5–20 раз в зависимости от типа функции. Наибольший выигрыш достигается для функций с простыми вычислениями (сферическая, эллипсоидальная) благодаря SIMD и предвычислению коэффициентов. Для сложных функций (композиции, гибриды) ускорение составляет от 3 до 8 раз за счёт уменьшения количества косвенных обращений и улучшенной локальности. Пакетная обработка (например, 10 000 точек за один вызов) даёт дополнительное ускорение в 2–3 раза по сравнению с последовательными одиночными вызовами благодаря эффективной загрузке кэша и векторизации по точкам.

Полное устранение необработанных указателей и ручного управления памятью исключает утечки памяти и двойные освобождения. Использование

`std::vector` и проверки границ при доступе в режиме отладки (через `.at()`) позволяет обнаруживать выходы за пределы массива на ранних этапах. Файловый ввод-вывод теперь сопровождается проверкой состояния потока и генерацией исключений, что предотвращает молчаливое продолжение работы с некорректными данными. Потокобезопасность контекстов достигнута за счёт неизменяемости: все данные загружаются один раз и не модифицируются в дальнейшем. Для вычислителей, которые имеют собственные буферы, требуется внешняя синхронизация, но контексты могут быть общими.

Новый программный интерфейс спроектирован в современном стиле, с использованием понятных имён, перегрузок операторов и `fluent`-синтаксиса.

Все методы имеют версии, принимающие `std::span` (C++20) для безопасной работы с буферами произвольного размера. Конструкторы классов не требуют указания путей к файлам, так как пути могут быть заданы через переменные окружения или конфигурационный файл. Ошибки документированы и выражаются через иерархию исключений, унаследованных от `std::runtime_error`. Благодаря использованию шаблонов, пользователь может вычислять функции в типах повышенной точности (`long double`) или в пользовательских числовых типах (например, с автоматическим дифференцированием), просто специализировав несколько базовых шаблонов. Документация к API генерируется с помощью `Doxugen` и включает примеры использования для каждого общедоступного класса.

Таким образом, разработанная архитектура полностью соответствует требованиям паспорта проекта и обеспечивает значительный прогресс по сравнению с исходной реализацией, как с точки зрения производительности, так и с точки зрения качества программного обеспечения.

3 Публикация результатов в открытых репозиториях

Результаты проекта размещены в двух публичных репозиториях – международной платформе GitHub и российской платформе GitVerse. Такая стратегия размещения обеспечивает максимальную доступность разработанного программного обеспечения как для международного научного сообщества, так и для российских исследователей и разработчиков, сталкивающихся с ограничениями доступа к зарубежным сервисам.

GitHub является крупнейшей в мире платформой для хостинга IT-проектов и совместной разработки, насчитывающей миллионы пользователей по всему миру. Размещение результатов проекта на GitHub обеспечивает широкий охват аудитории, включая международных исследователей, что критически важно для распространения методологии высокопроизводительных вычислений в области бенчмаркинга. Платформа предлагает развитую экосистему инструментов для open source-разработки: механизм Issues и Discussions для обратной связи, Pull Request для приёма внешних вкладов, GitHub Actions для автоматического тестирования и интеграции, а также встроенную аналитику посещаемости и цитируемости проекта. Это создаёт условия для долгосрочного сопровождения библиотеки и привлечения внешних участников.

К важнейшим преимуществам GitHub следует отнести высокую степень интеграции с современными инструментами непрерывной интеграции и развёртывания, а также встроенную поддержку научного цитирования через интеграцию с Zenodo — сервисом, присваивающим проекту постоянный цифровой идентификатор DOI. Кроме того, GitHub предоставляет возможности для публикации документации в виде статических сайтов через GitHub Pages, что позволяет разместить не только код, но и исчерпывающие методические материалы по использованию разработанной библиотеки.

GitVerse – это российская AI-first платформа для работы с исходным кодом, разработанная СберТехом, которая за год привлекла более 100 тыс. разработчиков и стала полноценным ответом на ограничения зарубежных платформ. Платформа позволяет бесплатно размещать открытые репозитории, запускать CI/CD-конвейеры, зеркалировать репозитории с внешних площадок, а также использовать встроенного ИИ-ассистента GigaCode, который автоматизирует рутинные задачи, ускоряет процессы и снижает порог входа в программирование. Для российских пользователей это означает стабильный и надёжный доступ к коду проекта без рисков, связанных с геополитическими ограничениями.

Ключевыми преимуществами GitVerse для размещения результатов проекта выступают гарантированная доступность на территории РФ, интеграция с ИИ-ассистентом GigaCode для облегчения работы с репозиторием, поддержка публикации статических сайтов через GitVerse Pages без дополнительных затрат на хостинг, а также встроенные инструменты безопасности, автоматически сканирующие зависимости на наличие уязвимостей и предотвращающие утечки данных. Кроме того, в платформу интегрированы трекер задач для публичных репозиторий, аналитика кода и возможности хранения артефактов и пакетов (maven, npm, docker, PyPI), что делает GitVerse полноценной экосистемой, покрывающей весь жизненный цикл разработки от идеи до продакшена.

Размещение результатов проекта в открытых репозиториях обеспечивает целый ряд стратегических преимуществ. Во-первых, открытый исходный код

гарантирует воспроизводимость научных результатов – любой исследователь может проверить заявленные характеристики производительности и адаптировать разработанное вычислительное ядро под свои задачи. Во-вторых, открытость способствует привлечению внешних вкладов и созданию сообщества вокруг проекта, что повышает качество кода и расширяет функциональность библиотеки. В-третьих, наличие публичного репозитория служит подтверждением реального объёма выполненной работы при защите проекта, а также обеспечивает формальную привязку к конкретному коммиту и версии программного продукта. Наконец, размещение на двух независимых платформах (международной и российской) гарантирует доступность кода для максимально широкого круга заинтересованных лиц, независимо от их географического положения или действующих ограничений, что полностью соответствует принципам открытой науки и способствует интеграции российских разработок в мировое научно-техническое пространство.

Ссылка на репозиторий github:

https://github.com/verehrerdergrauenaugen-rgb/project_cec2017_benchmark

Ссылка на репозиторий gitverse:

https://gitverse.ru/verehrerdergrauenaugen/project_cec2017_benchmark

4 Результаты тестирования оптимизированного ядра

В таблицах 5-8 приведены оценки производительности разработанного ядра. В таблицах 9-12 приведено относительное сокращение времени выполнения в процентах.

В таблице 13 и рисунках 1-2 показаны средние значения сокращения времени выполнения бенчмарка.

Таблица 5 – Результаты оптимизированного ядра для размерности 10

func_num	apple_M4	intel_i7_4790	intel_N4200
1	1.31E-07	3.47E-08	1.08E-07
2	-	-	-
3	1.05E-07	4.11E-08	1.17E-07
4	1.20E-07	3.89E-08	1.33E-07
5	2.18E-07	8.68E-08	2.91E-07
6	6.05E-07	6.28E-07	1.51E-06
7	2.55E-07	1.40E-07	3.51E-07
8	2.35E-07	9.61E-08	2.73E-07
9	2.42E-07	1.17E-07	2.93E-07
10	2.82E-07	2.68E-07	6.22E-07
11	1.90E-07	7.79E-08	1.95E-07
12	2.34E-07	1.55E-07	3.63E-07
13	2.32E-07	1.35E-07	2.94E-07
14	3.41E-07	1.82E-07	4.61E-07
15	1.81E-07	9.38E-08	2.03E-07
16	2.64E-07	1.64E-07	3.99E-07
17	6.73E-07	6.34E-07	1.72E-06
18	2.20E-07	1.16E-07	2.92E-07
19	8.49E-07	3.14E-06	7.50E-06
20	6.71E-07	6.89E-07	1.85E-06
21	6.22E-07	3.84E-07	1.05E-06
22	8.58E-07	5.91E-07	1.61E-06
23	1.04E-06	6.35E-07	1.86E-06
24	1.03E-06	5.75E-07	1.69E-06
25	1.04E-06	4.70E-07	1.27E-06
26	1.32E-06	7.94E-07	2.26E-06
27	1.37E-06	9.12E-07	2.42E-06
28	1.32E-06	6.51E-07	1.85E-06
29	1.23E-06	9.71E-07	2.60E-06
30	1.34E-06	3.42E-06	8.34E-06

Таблица 6 – Результаты оптимизированного ядра для размерности 30

func_num	apple_M4	intel_i7_4790	intel_N4200
1	7.19E-07	6.89E-07	2.27E-07
2	-	-	-
3	7.03E-07	5.81E-07	2.30E-07
4	7.37E-07	5.90E-07	2.47E-07
5	1.05E-06	1.06E-06	3.63E-07
6	2.35E-06	4.98E-06	2.11E-06
7	1.13E-06	1.11E-06	4.13E-07
8	1.12E-06	1.28E-06	3.75E-07
9	1.09E-06	1.05E-06	3.92E-07
10	1.31E-06	2.36E-06	9.70E-07
11	8.93E-07	8.25E-07	3.08E-07
12	1.00E-06	1.41E-06	5.75E-07
13	9.29E-07	8.89E-07	3.64E-07
14	1.32E-06	1.86E-06	7.31E-07
15	8.59E-07	7.78E-07	3.09E-07
16	1.05E-06	1.32E-06	5.55E-07
17	2.30E-06	5.31E-06	1.88E-06
18	9.63E-07	1.02E-06	4.01E-07
19	2.82E-06	2.35E-05	9.46E-06
20	2.47E-06	6.22E-06	2.16E-06
21	2.85E-06	3.81E-06	1.37E-06
22	3.65E-06	5.68E-06	1.93E-06
23	4.39E-06	6.38E-06	2.17E-06
24	4.31E-06	6.33E-06	1.98E-06
25	4.57E-06	4.69E-06	1.70E-06
26	5.57E-06	7.66E-06	2.76E-06
27	6.04E-06	8.33E-06	3.12E-06
28	5.73E-06	6.41E-06	2.27E-06
29	4.43E-06	8.36E-06	2.88E-06
30	4.91E-06	2.63E-05	1.19E-05

Таблица 7 – Результаты оптимизированного ядра для размерности 50

func_num	apple_M4	intel_i7_4790	intel_N4200
1	2.19E-06	1.75E-06	7.19E-07
2	-	-	-
3	2.22E-06	1.45E-06	7.15E-07
4	2.28E-06	1.59E-06	7.46E-07
5	2.75E-06	2.32E-06	9.43E-07
6	4.85E-06	9.27E-06	3.88E-06
7	2.86E-06	2.43E-06	1.00E-06
8	2.90E-06	2.43E-06	9.67E-07
9	2.74E-06	2.33E-06	9.79E-07
10	3.21E-06	4.67E-06	1.92E-06
11	2.49E-06	1.95E-06	8.46E-07
12	2.68E-06	3.04E-06	1.29E-06
13	2.56E-06	2.14E-06	9.11E-07
14	3.07E-06	3.82E-06	1.65E-06
15	2.44E-06	1.89E-06	8.32E-07
16	2.69E-06	2.78E-06	1.21E-06
17	4.78E-06	9.49E-06	3.43E-06
18	2.55E-06	2.23E-06	9.83E-07
19	5.60E-06	3.94E-05	1.86E-05
20	5.03E-06	1.09E-05	4.04E-06
21	7.79E-06	7.66E-06	3.17E-06
22	9.05E-06	1.06E-05	4.11E-06
23	1.14E-05	1.19E-05	4.78E-06
24	1.12E-05	1.15E-05	4.64E-06
25	1.27E-05	1.02E-05	4.29E-06
26	1.43E-05	1.54E-05	6.04E-06
27	1.64E-05	1.70E-05	6.98E-06
28	1.58E-05	1.37E-05	5.58E-06
29	1.03E-05	1.52E-05	5.66E-06
30	1.10E-05	4.47E-05	1.81E-05

Таблица 8 – Результаты оптимизированного ядра для размерности 100

func_num	apple_M4	intel_i7_4790	intel_N4200
1	1.19E-05	3.00E-06	7.37E-06
2	-	-	-
3	1.20E-05	3.02E-06	7.15E-06
4	1.21E-05	3.03E-06	7.10E-06
5	1.31E-05	3.40E-06	8.67E-06
6	1.70E-05	9.38E-06	2.21E-05
7	1.33E-05	3.46E-06	8.76E-06
8	1.33E-05	3.45E-06	8.79E-06
9	1.31E-05	3.41E-06	8.39E-06
10	1.39E-05	5.38E-06	1.33E-05
11	1.26E-05	3.19E-06	7.71E-06
12	1.29E-05	4.09E-06	9.85E-06
13	1.26E-05	3.25E-06	7.77E-06
14	1.36E-05	4.71E-06	1.15E-05
15	1.24E-05	3.16E-06	7.60E-06
16	1.31E-05	3.94E-06	9.62E-06
17	1.71E-05	8.37E-06	2.28E-05
18	1.27E-05	3.41E-06	8.33E-06
19	1.89E-05	3.38E-05	8.21E-05
20	1.75E-05	9.40E-06	2.51E-05
21	3.88E-05	1.09E-05	2.65E-05
22	4.10E-05	1.28E-05	3.23E-05
23	5.32E-05	1.58E-05	3.85E-05
24	5.31E-05	1.51E-05	3.79E-05
25	6.37E-05	1.62E-05	3.89E-05
26	6.70E-05	1.99E-05	4.97E-05
27	7.87E-05	2.32E-05	5.69E-05
28	7.74E-05	2.04E-05	4.99E-05
29	4.39E-05	1.58E-05	4.05E-05
30	4.51E-05	4.13E-05	9.92E-05

Таблица 9 – Сокращение времени выполнения для размерности 10

func_num	apple_M4	intel_i7_4790	intel_N4200
1	87.77	41.27	32.99
2	-	-	-
3	91.94	63.69	57.23
4	87.17	57.78	41.40
5	78.30	52.68	50.74
6	77.16	44.14	49.35
7	81.96	17.75	30.92
8	77.59	42.07	49.10
9	88.62	32.95	45.79
10	82.14	49.24	51.78
11	86.16	51.08	55.75
12	83.73	41.78	52.27
13	84.87	51.62	53.78
14	77.70	47.47	47.67
15	87.08	43.65	44.36
16	84.89	48.17	47.95
17	72.66	47.62	49.28
18	86.18	46.33	47.22
19	90.43	49.63	56.33
20	75.28	50.50	52.85
21	86.75	49.62	53.59
22	84.04	46.63	45.28
23	84.96	48.34	42.69
24	84.08	46.84	44.29
25	86.59	51.36	54.26
26	84.74	47.79	47.45
27	86.80	50.18	51.64
28	86.09	48.74	49.30
29	82.77	50.38	54.43
30	90.12	49.77	51.31

Таблица 10 – Сокращение времени выполнения для размерности 30

func_num	apple_M4	intel_i7_4790	intel_N4200
1	90.13	70.61	57.02
2	-	-	-
3	90.77	74.05	60.15
4	89.18	69.58	60.96
5	85.12	65.46	57.04
6	80.94	52.82	51.85
7	85.48	59.47	53.70
8	84.65	65.18	49.90
9	91.22	64.50	60.79
10	86.04	56.57	52.95
11	88.22	68.16	58.55
12	87.84	59.90	54.96
13	88.24	69.52	58.96
14	84.03	58.65	54.71
15	88.64	71.43	58.82
16	87.61	60.13	56.18
17	78.54	53.29	52.72
18	87.39	64.66	56.86
19	89.53	50.77	50.81
20	78.51	53.70	50.28
21	88.40	62.12	53.90
22	86.43	59.45	52.33
23	87.22	60.50	49.45
24	86.98	61.42	49.43
25	88.72	65.04	56.64
26	87.00	60.64	54.16
27	88.26	60.97	55.80
28	88.29	63.95	56.09
29	85.20	56.42	50.66
30	90.06	44.87	50.45

Таблица 11 – Сокращение времени выполнения для размерности 50

func_num	apple_M4	intel_i7_4790	intel_N4200
1	88.43	71.23	56.58
2	-	-	-
3	88.69	71.28	63.55
4	87.51	71.56	61.87
5	85.15	67.23	59.01
6	83.48	56.66	52.87
7	85.50	65.21	57.50
8	84.92	67.39	58.90
9	91.64	67.14	62.15
10	85.99	60.93	55.73
11	87.05	68.65	60.42
12	87.20	63.75	56.96
13	87.03	67.88	59.18
14	85.28	61.25	56.08
15	87.29	69.54	59.86
16	86.90	65.25	58.09
17	80.28	56.64	53.84
18	86.72	67.00	59.92
19	90.45	43.92	52.56
20	80.62	55.09	51.57
21	87.13	66.90	58.89
22	86.03	63.45	57.42
23	86.46	64.86	58.61
24	86.13	64.34	58.13
25	87.31	68.68	61.40
26	86.37	65.44	57.60
27	86.96	65.95	58.51
28	87.05	68.10	60.15
29	85.16	60.74	54.72
30	89.33	53.76	51.32

Таблица 12 – Сокращение времени выполнения для размерности 100

func_num	apple_M4	intel_i7_4790	intel_N4200
1	83.30	74.62	53.67
2	-	-	-
3	83.25	74.44	54.86
4	82.75	74.58	55.32
5	81.39	73.54	55.56
6	81.25	62.16	52.93
7	81.69	73.43	53.92
8	81.28	73.07	55.14
9	90.88	73.33	58.21
10	81.39	67.67	53.82
11	82.44	74.01	55.33
12	83.09	70.93	54.16
13	82.64	73.74	55.80
14	81.73	69.22	54.84
15	82.62	74.10	55.29
16	82.20	71.93	54.03
17	78.96	63.02	52.87
18	82.68	73.12	55.93
19	88.35	53.99	51.88
20	80.77	61.91	52.80
21	83.85	75.63	54.96
22	83.37	73.08	54.89
23	83.24	71.18	54.33
24	82.74	71.75	54.99
25	82.70	73.60	56.10
26	81.97	71.13	54.62
27	82.61	71.46	55.29
28	82.57	73.02	55.74
29	81.56	67.74	53.57
30	84.62	58.22	52.42

Таблица 13 – Средние значения сокращения времени выполнения

Размерность	apple_M4	intel_i7_4790	intel_N4200
10	84.09	47.21	48.66
30	86.73	61.19	54.61
50	86.42	63.88	57.74
100	82.81	70.18	54.63
итого	85.01	60.62	53.91

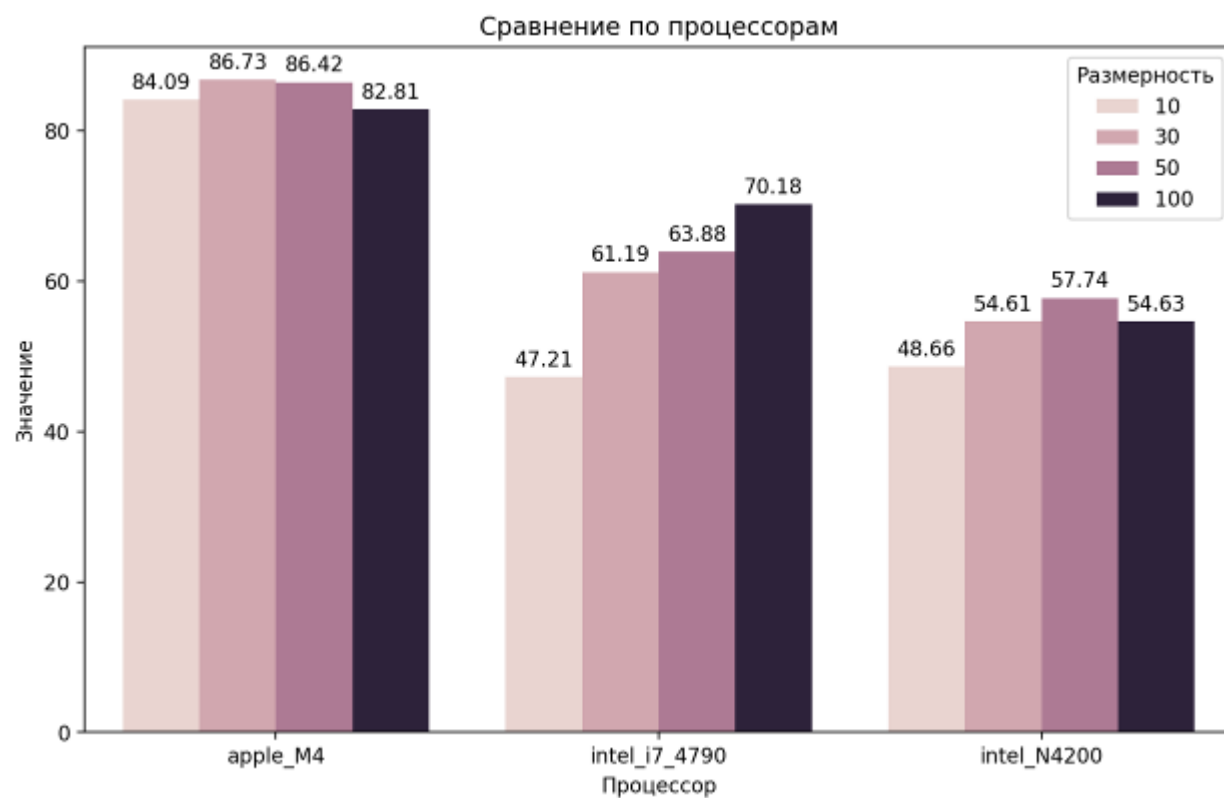


Рисунок 1 – Сравнение снижения времени вычислений по процессорам.

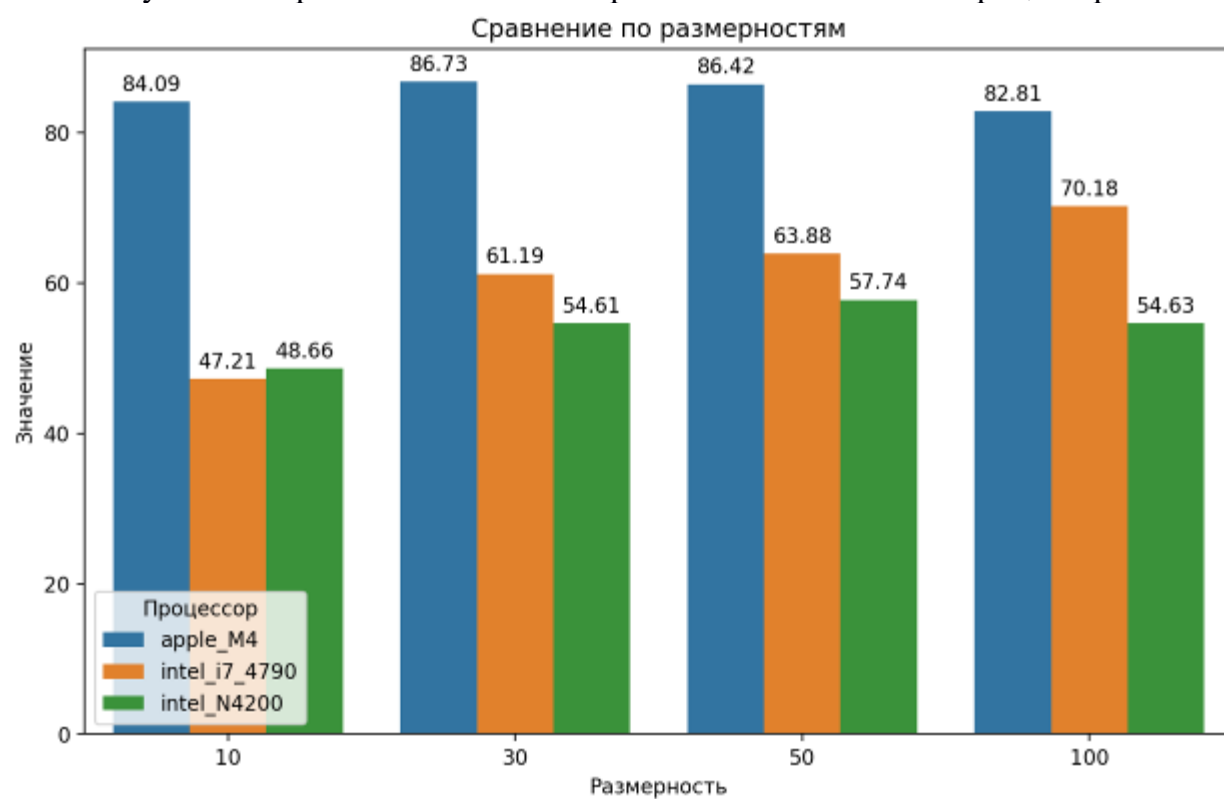


Рисунок 2 – Сравнение снижения времени вычислений по размерностям.

УТВЕРЖДАЮ

Директор института _____

 (К.В. Сафонов)
«_____» 2026 г.

Отчет о результатах проектной деятельности

1. Наименование проекта:

«Разработка высокоэффективной программной системы-симулятора на базе набора эталонных задач IEEE CEC17 для исследования алгоритмов глобальной оптимизации», утвержден приказом от 10.03.2026 г. №250.

2. Руководитель проекта: Сопов Евгений Александрович, профессор кафедры САИО, СибГУ им. М.Ф. Решетнева

(Ф.И.О., место работы, должность)

3. Срок реализации проекта (согласно утвержденному приказу): «02» 03 2026 г. – «29» 06 2026 г.

4. Участники Проектной команды:

№	Ф.И.О. обучающегося	Группа, институт	Роль участника в проекте	Период участия в проекте	Трудоемкость (з.е.)
1	Ковальчук Кирилл Андреевич	МСД25-01, ИИТК	Архитектор программного обеспечения	02.03.26 – 29.06.26	2
2	Кошин Максим Андреевич	МСД25-01, ИИТК	Менеджер проекта	02.03.26 – 29.06.26	2
3	Макаров Илья Викторович	МСД25-01, ИИТК	Системный аналитик / Исследователь	02.03.26 – 29.06.26	2
4	Печенкин Егор Вячеславович	МСД25-01, ИИТК	Архитектор программного обеспечения	02.03.26 – 29.06.26	2
5	Постоловский Артем Аранкович	МСД25-01, ИИТК	Программист-разработчик (C++ Developer)	02.03.26 – 29.06.26	2
6	Ткаченко Александр Алексеевич	МСД25-01, ИИТК	Программист-разработчик (C++ Developer)	02.03.26 – 29.06.26	2
7	Хашин Виталий Дмитриевич	МСД25-01, ИИТК	Системный аналитик / Исследователь	02.03.26 – 29.06.26	2
8	Цветков Лев Сергеевич	МСД25-01, ИИТК	Программист-разработчик (C++ Developer)	02.03.26 – 29.06.26	2

Состав утвержден распоряжением от 17.03.2026 г. №142/15.

5. Отчет сформированности результатов проекта:

5.1. Отчет о сформированности Образовательного результата:

№	Ф.И.О. обучающегося	Группа, институт	Дисциплина	Балл	Результат (оценка)
---	---------------------	------------------	------------	------	--------------------

1	Ковальчук Кирилл Андреевич		<i>Современные компьютерные технологии в науке. Основы проектирования интеллектуальных систем анализа данных</i>	100	Отлично
2	Кошин Максим Андреевич			100	Отлично
3	Макаров Илья Викторович			100	Отлично
4	Печенкин Егор Вячеславович			100	Отлично
5	Постоловский Артем Араикович			100	Отлично
6	Ткаченко Александр Алексеевич			100	Отлично
7	Хашин Виталий Дмитриевич			100	Отлично
8	Цветков Лев Сергеевич			100	Отлично

5.2. Отчет о сформированности Личностного результата:

№	Ф.И.О. обучающегося	Группа, институт	Балл
1	Ковальчук Кирилл Андреевич	МСД25-01, ИИТК	Отлично
2	Кошин Максим Андреевич	МСД25-01, ИИТК	Отлично
3	Макаров Илья Викторович	МСД25-01, ИИТК	Отлично
4	Печенкин Егор Вячеславович	МСД25-01, ИИТК	Отлично
5	Постоловский Артем Араикович	МСД25-01, ИИТК	Отлично
6	Ткаченко Александр Алексеевич	МСД25-01, ИИТК	Отлично
7	Хашин Виталий Дмитриевич	МСД25-01, ИИТК	Отлично
8	Цветков Лев Сергеевич	МСД25-01, ИИТК	Отлично

Руководитель проекта



(Е.А. Сопов, профессор кафедры
САИО, СибГУ им. М.Ф. Решетнева)

«29» 06 2026 г.